

# Counting Versus Testing the Destination: A Simulation Study of Layered Defences Against SMS OTP Flooding and Pumping

Naveed Ahmed<sup>a,\*</sup>

<sup>a</sup>*Affiliation to be completed before submission, Saudi Arabia*

---

## Abstract

Endpoints that send one-time passwords by SMS are abused to waste money (flooding) or to earn part of the termination fee (SMS pumping). We evaluate in simulation a twelve-step defence that draws trust from app attestation, signed sessions and verification outcomes, against simpler designs and 25 attacker profiles. Attacks with a signature in the request are contained at little cost to simulated users; a residential attacker with farmed CAPTCHA scores is rationed, not separated, by refusing half of first-time users. On the destination, short-window counters of SMS sends per number block, each selected at a benign service target for one traffic density, leaked 6 to 48 messages against four concentrated 20-minute pumpers at 0.06 to 0.43 points of attack-free completion; at the busiest density the default sequential tests of verification outcomes leaked 19 to 452. Under a second predeclared protocol, on seeds and shifted workloads unseen by selection, the frozen counter met its leakage and benign-cost claims, leaked no more than the tests in all but 13 of 144 cells, all against an instantly verifying carrier, and failed both service-under-attack claims when the pumper shares blocks with real users. A heuristic with constants from the configuration, not fitted, ordered the two policies correctly in all 18 boundary cells where its estimates differ: our tests leak about five messages a block plus those sent while early failures are unresolved, the counter its quota per block in every window, so it loses to paced pumpers and to wide, long campaigns.

*Keywords:* SMS pumping, artificially inflated traffic, one-time password,

---

\*Corresponding author.

*Email address:* [optimistbs@gmail.com](mailto:optimistbs@gmail.com) (Naveed Ahmed)

## 1. Introduction

Most online services verify a phone number by sending it a code by SMS, from an endpoint reachable before any account exists, at a tenth of a cent to twenty cents a message. A flooder makes the service send to numbers it does not own; a pumper sends to a range whose operator shares the termination fee, turning the service’s SMS budget into its income [1]. Twitter’s owner said in 2023 that pumping had cost the company about 60 million dollars a year [2], and Huh et al. found as many attack requests as genuine ones in three months of one large service’s logs [3].

This paper starts from such an incident on a registration flow the author operated, and from the redesign that followed: a twelve-step pipeline with attested platforms, signed sessions, number checks before a message is paid for, reputation from whether codes were entered, a five-response risk score and a hard budget. We ask in simulation which attacks it contains against simpler designs, at what cost in completed registrations (RQ1); which layer is responsible (RQ2); when outcome-based reputation separates an attacker from simulated users (RQ3); what rationing costs when it does not (RQ4); and what destination policies cost legitimate users, with and without an attack (RQ5).

The destination became the centre of the study, because a pumper is paid only on its partner’s ranges: under a predeclared comparison a plain counter of sends per number block leaked less than our sequential tests of code outcomes, and a second predeclared protocol mapped where that holds and fails. We contribute eight gaps recalled from the incident, as motivation only, the design and an open implementation, measurements of where reputation stops separating and what rationing depends on, both destination-policy comparisons with their failures and collateral damage, and a heuristic, checked against the simulation, for when counting leaks less than our tests; Sections 7.1–7.4 are context for the destination results (Sections 7.5–7.7).

## 2. Background and related work

*Revenue-share fraud.* In international revenue share fraud, traffic is generated to an advertised number range and the termination fee is split along

33 the route [4]. Sahin and Francillon’s 3.14 million test numbers from twelve  
34 providers show the range to be the fraudster’s managed asset [5].

35 *Detecting artificially inflated SMS.* Huh et al. trained gradient-boosted trees  
36 on 9.4 million SMS authentication logs from nineteen countries, reaching  
37 89.6% recall at 0.19% false positives on web traffic [3]. About 73% of web  
38 attacks sent four or fewer requests per identity a day, defeating per-entity  
39 limits, and destination signals were the strongest: about 92% zero conversion  
40 for attacks against 21% for genuine traffic. Their detector needs labelled  
41 traffic and acts per request on request, history and country features; our  
42 policies need no labels, act per destination block and see only send counts or  
43 code outcomes. Theirs was not reproduced, so the two are not ranked.

44 *Defences in practice.* Twilio’s Fraud Guard blocks regions known for pump-  
45 ing [6], and its SMS Pumping Risk Score advises friction from 60 and refusal  
46 above 90 [7], so graded destination action is established practice; no product  
47 was benchmarked. CAPTCHA solving is a labour market at about a dollar  
48 per thousand [8], open where machine solvers fail [9]; client puzzles [10] were  
49 not evaluated. Residential proxy addresses rotate within minutes [11], and  
50 carrier-grade NAT makes address reputation on cellular networks “all but  
51 infeasible” [12].

52 *Identity and change detection.* One entity can present many identities [13];  
53 a signed session gives continuity, not scarcity. Our block tests are Wald’s  
54 sequential tests [14] run as Page’s cumulative sums [15]. Quickest change de-  
55 tection minimises delay under false-alarm constraints [16], and a CUSUM’s  
56 parameters follow from the error rates it must meet [17]; windowed statistics  
57 and confidence sequences [18] were not evaluated, so ‘testing’ below means  
58 our conversion and speed tests only. Step 7 resembles risk-based authentica-  
59 tion [19]; attacks on the code once sent are a separate problem [20, 21].

### 60 3. The incident and the adapted attacker

61 This section is motivation, not evidence: no logs from the period are in  
62 our hands, permission to describe the incident is being sought, and no result  
63 depends on it. The flow sent an SMS to any unregistered number. The  
64 first controls, v1, were reCAPTCHA v3 on web requests, one message per  
65 number per minute, five requests per address per minute, a country allow-  
66 list, a body-length limit, a proxy block and per-platform caps. The returning

67 abuse used thousands of residential addresses, carried the iOS header to skip  
 68 the CAPTCHA, and stayed under every limit.

Table 1: The v1 controls and their gaps, recalled by the operator from the incident (Recalled; not supported by logs) or found in the design review that followed (Review).

| Gap | Source   | v1 assumed                                             | How it fails                                |
|-----|----------|--------------------------------------------------------|---------------------------------------------|
| A   | Recalled | A platform header says which app sent the request      | Sending the header reached every exemption  |
| B   | Recalled | Five requests per address per minute bound an attacker | A fresh residential address per request     |
| C   | Recalled | A number given is worth sending to                     | Generated numbers cost as much as real ones |
| D   | Review   | An allowed country is a safe destination               | Premium ranges inside it cost far more      |
| E   | Recalled | Per-key caps bound total spend                         | A wide attack under every cap               |
| F   | Review   | Check-then-increment is a rate limit                   | Two racing instances both pass              |
| G   | Review   | Bulk and test flags are internal                       | Neither needs stronger authentication       |
| H   | Review   | Each check says which check failed                     | Thresholds can be probed one at a time      |

69 *Threat model.* The attacker wants as many messages sent as possible, or  
 70 sent to numbers that pay it. It buys residential proxies and human-like  
 71 reCAPTCHA tokens (about 0.3 cents each), mints fingerprints, generates  
 72 numbers and knows the thresholds. A pumper also has a colluding operator's  
 73 ranges, where it may enter codes, report deliveries as failed, build verified  
 74 history, spread over blocks and buy challenge solutions. It cannot forge an  
 75 attestation, use ranges its partner does not terminate, or make a stranger  
 76 enter a code.

## 77 4. Design

78 No message is paid for before every check passes, and cheap checks run  
 79 first. Trust comes from evidence the client cannot forge, an attestation, a  
 80 server-signed session and the fate of earlier codes, though such evidence can  
 81 be manufactured (Section 7.8).

82 *4.1. The pipeline and the risk score*

83 A request passes the steps of Table 2 until one refuses it. Step 0 takes  
 84 the platform only from a valid App Attest or Play Integrity result. Each  
 85 limit and reservation is one atomic Redis operation, though the enclosing  
 86 steps also make separate writes (Section 6). Per-number claims and the per-  
 87 block counter slot are released if a later step refuses, and the hourly budget  
 88 is reserved only for a message that is sent; address limits and source caps  
 89 count requests, except that a request Step 11 challenges gives its source-cap  
 90 count back. Every outcome after Step 0 except the challenge returns one  
 91 HTTP 200 body after at least 400 ms, a floor that cannot hide longer work.

Table 2: The twelve steps (the v1 gaps of Table 1 they address in brackets). FB is the feedback loop.

| Step | Check                                                                                                    |
|------|----------------------------------------------------------------------------------------------------------|
| 0    | Proxy block; platform from attestation, never from the header [A]                                        |
| 1    | Signed session bound to a fingerprint; 3 requests per session per 10 min [B]                             |
| 2    | Limits per address, /24 and ASN, lower for hosting ASNs [B, F]                                           |
| 3–4  | reCAPTCHA v3 threshold and origin check, web only                                                        |
| 5    | Number intelligence: cost class, sequential numbers, liveness; reads the block’s verdict stage [C, D, G] |
| 6    | Body length, excluded numbers; test bypass needs a credential [G]                                        |
| 7    | Risk score and tier: allow, delay, challenge, downgrade, block [H]                                       |
| 8    | Per-number backoff, 60 s doubling to 1 h, five a day                                                     |
| 9    | Per-source, platform and country caps with an adaptive multiplier                                        |
| 10   | Operating mode: elevated above 80% of the hourly budget, emergency at 100% [E]                           |
| 11   | Reserve hourly count, spend and any destination slot; write state; enqueue; uniform response [E, H]      |
| FB   | Outcomes per key; destination-block tests; denylist of fine keys                                         |

92 Step 7 computes

$$s = \sum_i w_i x_i + w_c \frac{\max(0, \theta - r_{\min})}{\theta} + w_s f + w_m i - w_v v_n - w_f v_f, \quad (1)$$

93 where the  $x_i$  are request signals (CAPTCHA score, hosting ASN, fingerprint  
 94 age, session requests, country mismatch, number class),  $r_{\min}$  is the lowest con-  
 95 version ratio over the request’s keys (address to destination block),  $\theta = 0.3$ ,  
 96  $w_c = 25$ ,  $f$  and  $i$  flag a flooded key and one whose codes are entered within

seconds, and  $v_n$ ,  $v_f$  mark a number and fingerprint with verified history; other weights are 5 to 25 [22]. Tiers start at 20, 40, 60 and 80: *allow*; *delay* (a doubling wait); *challenge* (credits  $-20$  when solved); *downgrade* (push or WhatsApp, never SMS); *block*.

#### 4.2. The feedback loop

A send resolves as *verified* when its code is entered, *failed* after five wrong attempts or two minutes past the delivery receipt, and *undelivered* without a receipt in sixty seconds or with a failure receipt; only the first two feed reputation. A positive receipt after a negative one reopens the send; a late verification reclassifies it, but an issued verdict stands. Each transition records its effects and its time in the same compare-and-set, and a recovery sweep finishes a dead process's work; each effect type applies at most once, and exactly once if the sweep runs within 20 minutes, at the transition's time: reputation counts land in its hour, a block keeps its latest outage failure time, and a verdict is dated at the event that triggers it.

#### 4.3. Destination policies

A pumper is confined to its partner's ranges but can rotate numbers and blocks within them. Destination policies are keyed on the 8-digit block (10 000 numbers), so their protection weakens as a pumper spreads.

*Sequential tests.* The conversion statistic assumes users verify with  $p_\ell = 0.8$  and a flooder with  $p_a = 0.1$ : a failure adds  $\ln \frac{1-p_a}{1-p_\ell} = 1.50$ , a verification  $\ln \frac{p_a}{p_\ell} = -2.08$ . The speed statistic assumes a machine enters a verified code within five seconds with  $q_a = 0.9$  and a person with  $q_\ell = 0.2$ , since autofill removes the keystroke [23, 24]; it adds 1.50 or  $-2.08$ . A verdict is issued, and the statistic reset, when either exceeds  $h = \ln 1000 = 6.91$ . The floor  $S_t = \max(-ch, S_{t-1} + \text{increment})$  sets the memory: unbounded  $c$  is Wald's test,  $c = 0$  Page's cumulative sum; the default is  $c = 1$ . A first verdict makes the block's first-time clients pass a challenge for an hour; a second within the hour moves them off SMS, refusing those without another channel.

*Counters.* A counter limits SMS sends per block in a daily or short refilling window. Step 11 reserves a slot, and decides the stage, in one atomic operation when an SMS is about to go out, so a refused or challenged request costs nothing and concurrent requests cannot skip the challenge. Past the limit a graded counter challenges first-time clients, and past twice the limit

131 moves them off SMS; a refusing counter refuses. Clients with verified history  
 132 are exempt from both policies, so a code entered on a pumper’s number can  
 133 buy that number an exemption. A token bucket of the same rate (burst 4 or  
 134 12; its challenge-tier bucket refills independently, whereas RFC 2697’s excess  
 135 bucket fills only from the committed one [25]) and the counter combined with  
 136 the tests were explored as alternatives.

#### 137 4.4. Capacity controls

138 Step 9 multiplies its caps by 0.25 to 1.5 according to a baseline job com-  
 139 paring each key with its expected rate, learned from the pipeline’s hourly  
 140 counters as a weekly profile or from the closed hours available; with none,  
 141 the static cap stands. Verified history avoids the adaptive reduction, not the  
 142 static caps or budget, and the budget bounds SMS count and spend units,  
 143 not vendor or fallback costs.

### 144 5. Leakage estimates and a heuristic crossover

145 We compare the SMS sent to a pumper’s numbers over an attack of  $T$   
 146 minutes from a cold start, by a pumper that solves no challenge and holds  
 147 no verified history. Against the sequential tests, one reaching  $B$  blocks with  
 148  $N$  requests at rate  $\lambda$  leaks about

$$L \approx \min(N, kB + \lambda\tau), \quad k = \lceil h/1.50 \rceil = 5, \quad (2)$$

149 before its first verdict, where  $\tau$ , the time a send stays unresolved, is the  
 150 resolution timeout plus the delivery delay plus half the timeout worker’s  
 151 period (2.55 minutes) if the carrier never verifies, and seconds if it verifies at  
 152 once. Judged, such a pumper is held for the verdict’s hour, so for  $T \leq 60$   
 153 Equation (2) also estimates the attack’s total; it ignores prior credit and  
 154 outage suspension. A carrier verifying a share  $\sigma$  of codes with human-like  
 155 delay gives the conversion statistic an expected increment of  $\sigma(-2.08) + (1 -$   
 156  $\sigma)(1.50)$ , zero at  $\sigma^* = 0.42$ ; above it the statistic drifts away from a verdict.

157 A counter of  $q$  sends per block opens a window with a block’s first reserved  
 158 SMS and closes it after the window length, so the pumper sends at most

$$L_c \leq \min(N, qBw), \quad w = \lceil T/\text{window} \rceil, \quad (3)$$

159 whatever its carrier does with the codes; solving challenges doubles the  
 160 bound, and verified history exempts a client from it. Both estimates grow

161 with  $B$ . While neither reaches  $N$ , the counter’s bound lies below the tests’  
 162 estimate when

$$B(qw - k) < \lambda\tau, \quad (4)$$

163 a heuristic crossover between a bound and an approximation, not a necessary  
 164 condition: other limits can hold either policy below its estimate. A carrier  
 165 that games outcomes may never be judged, so the tests’ leakage approaches  
 166  $N$ .

167 In the code every send takes a slot, verified clients’ included. If a block’s  
 168 sends arrive as a Poisson stream with mean  $m$  per window and all reach  
 169 the counter, a window holds  $1 + X$  of them,  $X \sim \text{Poisson}(m)$ , and the share  
 170 beyond  $q$ , which a graded counter challenges, is  $E[(1 + X - q)^+]/(1 + m)$ : 7.3%  
 171 at  $m = q/2$  and 27% at  $m = q$  for  $q = 4$ . This idealization ignores challenge  
 172 failures, which take no slot, and earlier gates. The forms of Equations (3)  
 173 and (4) and  $k$  follow from the rules,  $\tau$  from the configuration and  $\lambda$  from the  
 174 attack; what an excess send costs a user needs the simulation.

## 175 6. Implementation and evaluation method

176 *Implementation.* About 4000 lines of Python implement the pipeline, feed-  
 177 back loop and vendor adapters on in-memory and Redis backends. Release  
 178 2.8.0 (commit 1c277a0) produced the results. Releases 2.8.1 to 2.8.3 repair  
 179 seven defects outside the simulation: four in replay timing (an old failure  
 180 re-entering the outage window; an older failure moving its block’s latest fail-  
 181 ure time back; block-test events and verdicts taking the recovery time; a  
 182 reversal applied to the wrong test), a late reversal recovered after thirteen  
 183 minutes, a correcting receipt during the timeout worker’s transition, and  
 184 source caps kept by Step 11 challenges. Each has a regression test that fails  
 185 on the release before, and 102 recorded runs, three per study, replay identi-  
 186 cally under the repaired code. Of 436 tests, eighteen run two instances on  
 187 a real `redis-server`, testing specific interleavings, not all. The simulator’s  
 188 in-memory store, unlike Redis, lets a sorted set expire from its first write,  
 189 shortening the outage detector’s and Step 5’s memories; with Redis’s rule 27  
 190 of the 102 replayed runs change, 16 only in outage alerts and four in leakage  
 191 (one tuning run: 268 messages against 21). Step 11 is still not one transac-  
 192 tion: a crash after its reservations leaks reserved slots or resolves the send as  
 193 undelivered. The implementation is a research prototype.



194 *Simulation.* The pipeline runs against scripted vendor fakes on a simulated  
 195 clock. After a warm-up of ten minutes (30 or 130 for pumping), attacks  
 196 run over legitimate traffic at 20 requests per minute. Simulated users verify  
 197 80% of codes after a lognormal delay (median 22s; 2s for the 20% using  
 198 autofill); 70% of numbers are reachable on WhatsApp; 90% of users complete  
 199 a challenge; 20% are returning account holders with a stable fingerprint, a  
 200 trusted number and verified history; a returning person without them is, to  
 201 the pipeline, a first-time user, so results for returning users describe trusted  
 202 identities only. Separate random streams give all scenarios at a seed the same  
 203 users.

204 Three behavioural parameters have published counterparts: genuine con-  
 205 version (one provider’s 68% or more [26]; ours 60 to 90%) and the never-  
 206 converting destinations and identity churn of Section 2, which our non-  
 207 verifying pumpers and one-use farm share; the rest are assumptions. At-  
 208 tackers (Table 4) draw pool size, rate and CAPTCHA class per seed; in  
 209 a nested design (ten configurations, three seeds each) these draws account  
 210 for 92% [80, 98] of the variance of the farm’s leakage. Brackets are 95%  
 211 percentile-bootstrap intervals, paired per seed for differences; they describe  
 212 variation under this generator, not uncertainty about real traffic.

213 *Two protocols and their chronology.* Table 3 maps the studies. The first  
 214 protocol, committed before its runs, fixed the matched comparison and a  
 215 robustness study over twelve Latin-hypercube points (conversion 0.6–0.9 and  
 216 five other behavioural parameters) and a held-out family (60 to 120 attack  
 217 requests a minute, 50 000 to 500 000 addresses, heterogeneous legitimate traf-  
 218 fic), with claims C1–C5 that hold if true in 90% of cells, a benchmark con-  
 219 vention, not a confidence level. The matched comparison changes only the  
 220 destination policy and, at three legitimate block densities, selects on tuning  
 221 seeds the least leakage summed over four concentrated pumpers among set-  
 222 tings at or above the *tuning target*, the default’s completion minus 0.5 points  
 223 (the full sequential grid only at the busiest). The robustness study’s first run  
 224 failed C1 in 32 of 108 cells because attack and no-attack runs drew differ-  
 225 ent users; we fixed the pairing, a fix that followed a result, and reran every  
 226 study. The second protocol (claims K1–K5) was written after the matched  
 227 comparison had selected three counters and committed before its runs; six  
 228 of its seed values had served other robustness points, never selection. Both  
 229 ran as clean invocations (9 919 and 7 925 runs, each carrying the code hash);  
 230 300 later runs use 2.8.1.

Table 3: The studies, their seeds (three per shifted point), attack durations and results. MC: Monte Carlo.

| Study                                      | Seeds                   | Attack              | Results                        |
|--------------------------------------------|-------------------------|---------------------|--------------------------------|
| Designs; ablation, interactions            | 0–29; 0–9               | 20 min              | Table 5; Figure 1; Section 7.2 |
| Dilution; score and cap sweeps; cadence    | 0–4; 0–9                | 60; 20 min          | Sections 7.3, 7.4              |
| Pumping and spread                         | 0–9                     | 20 (60) min         | Table 6; Section 7.5           |
| False alarms; poisoner; adaptive attackers | MC; 0–4, 0–7, 0–9       | 24 h; 20–70 min     | Sections 7.7, 7.8              |
| Protocol 1: matched comparison             | tuning 100–102; 0–9     | 20 min; 24 h benign | Table 7; Figures 2, 3          |
| Protocol 1: robustness, C1–C5              | 12 points               | 20–60 min           | Sections 7.1–7.7               |
| Protocol 2: E1 attacked service            | 300–309                 | 60 min              | Table 9                        |
| E2 held-out family, K1–K5                  | 12 points               | 20 min              | Table 8                        |
| E3 adaptive; E4 boundary                   | 300–309; 300–304        | 20–360 min          | Section 7.6; Figure 4          |
| E5 selection, every density                | tuning 100–102; 300–309 | 20 min; 24 h benign | Section 7.6                    |

231 *Designs and metrics.* Seven designs: **v1** (header-trusted platform, static  
 232 caps), **v1-corrected** (attested platform), **budget-only** (sessions, static  
 233 caps, budget), **block-limit** (a refusing counter of five sends per block per  
 234 day), **conversion-only** and **speed-only** (v2 minus one block test), and **v2**.  
 235 A seed is *contained* when leakage stays at most 5% of the attack rate from  
 236 some minute to the window’s end, for at least five minutes. *Attributable* loss  
 237 compares the same requests with the policy enforced and absent, net of the  
 238 reverse, in points of offered users: on attack-free traffic it is the *benign cost*,  
 239 on the attacked trace the *extra harm* relative to the attacked no-policy arm.  
 240 *Degradation* compares the attacked trace with attack-free operation under  
 241 the same policy.

Table 5: Leaked share (% , mean of per-run shares) of about 600 attack requests in twenty minutes, thirty seeds, with source caps lifted (v1, block-limit BL, v2) and on (v2, with the first-time users refused and all users completing, %). v1-corrected as v1 except on the forged header (99.5); budget-only as v1-corrected except on the single client (0.3) and premium pumping (94.9); conversion-only as v2; speed-only as v2 except on the walk (13.3; caps on 12.3, 3.8, 77.3). Aged fingerprints behave as the farm.

| Attacker            | caps lifted: leaked |      |      | caps on, v2 |         |           |
|---------------------|---------------------|------|------|-------------|---------|-----------|
|                     | v1                  | BL   | v2   | leaked      | refused | completed |
| Single client       | 5.6                 | 0.3  | 0.3  | 0.3         | 1.1     | 79.1      |
| Datacenter rotation | 69.0                | 69.0 | 0.0  | 0.0         | 1.1     | 79.1      |
| Premium pumping     | 99.4                | 1.0  | 0.0  | 0.0         | 1.1     | 79.1      |
| Sequential numbers  | 99.4                | 1.0  | 11.5 | 10.7        | 3.5     | 77.5      |
| Reused profile      | 99.5                | 99.5 | 22.5 | 17.4        | 8.6     | 74.2      |
| Residential bot     | 37.2                | 37.2 | 37.2 | 29.9        | 16.2    | 69.6      |
| Farmed CAPTCHA      | 99.5                | 99.5 | 99.5 | 45.6        | 54.0    | 45.5      |
| Forged header       | 100.0               | 99.5 | 99.5 | 45.6        | 54.0    | 45.5      |

Table 4: The 25 attacker profiles by capability, and the destination policies each faced: S, sequential tests; C, counters; B, block-limit. Names and parameters: **results/attacker\_profiles.md**.

| Capability                          | Profiles                                                                                                                                      | Policies |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|----------|
| A signature in the request (5)      | One address and session; datacenter rotation abroad; a premium-rate prefix; numbers walked upward; the farm with an unattested iOS header     | S, B     |
| Looks like a person (7)             | Residential bot; farmed CAPTCHA with fresh or aged fingerprints; one 48-hour-old browser profile                                              | S, B     |
|                                     | Low and slow, 2 or 10 requests/min over 20 ASNs; datacenter rotation buying challenge solutions                                               | S        |
| Games outcomes or trust (8)         | Pumpers on elevated-cost or standard ranges verifying within 1 s, or 60% after 30 s                                                           | S        |
|                                     | Threshold-aware and receipt-faking carriers; three trust builders (500 identity/number pairs verify for 10 or 30 min, then flood)             | S, C     |
| Concentrated on the destination (5) | Four pumpers on three blocks (carrier never verifies, verifies within 1 s or 60% after 30 s, or buys challenges); a poisoner on users' blocks | S, C     |
| Spread (generated)                  | 3 to 300 blocks, at full rate or at most the counter's quota per block                                                                        | S, C     |

## 242 7. Results

### 243 7.1. RQ1: what is contained, and at what cost

244 Datacenter rotation leaks nothing under v2 because it scores in the chal-  
245 lenge tier and declines challenges; Step 5 refuses the premium range, and the  
246 session cap holds the single client to two messages (Table 5, Figure 1); in all

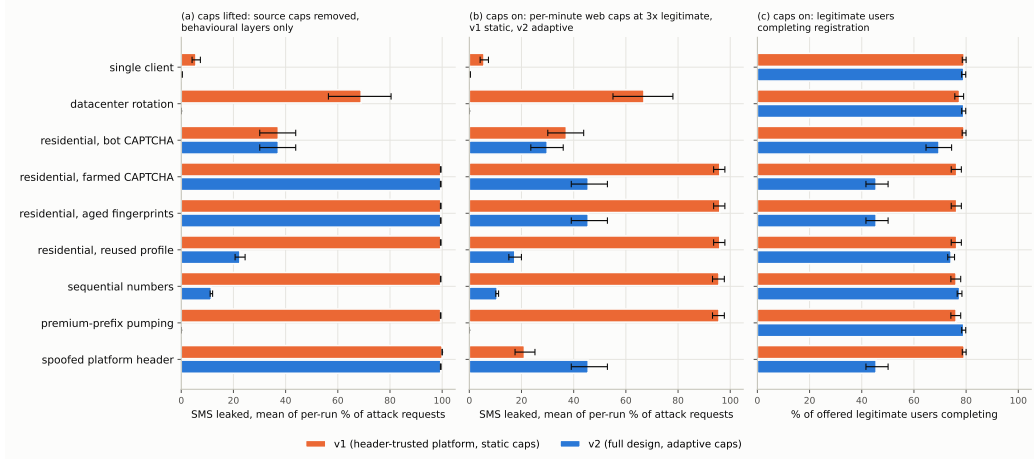


Figure 1: Leaked share of attack requests under v1 and v2: (a) source caps lifted, behavioural layers only; (b) per-minute source caps at three times the legitimate rate; (c) legitimate completion with caps on. Thirty seeds, 95% bootstrap intervals.

108 robustness cells these attacks leaked under 5% at completion within two points of the paired no-attack run (claim C1). The walk leaks 61 [53, 70] messages before its block’s verdict (block-limit: five), the reused profile 88 [83, 95].

The farm leaks 99.5% with caps lifted under every design; v2’s adaptive cap holds it to 229 [215, 244] messages, 46%, by refusing 54% [47, 60] of first-time users, so 45.5% complete instead of 76.4% under v1: rationing (claim C5 held in 36 of 36 cells).

### 7.2. RQ2: conditional effects of each layer

In the ablation grid (90 effects, each conditional on all other layers, with unadjusted intervals; artifact), removing the adaptive caps lets the farm leak 310 [194, 421] more messages; the risk engine, feedback loop and block key hold the walk by 166, 152 and 121 messages, and the loop holds the reused profile by 138. Effects overlap: the key acts only through the loop (interaction  $-121 [-141, -101]$ ), while the estimated interaction of the farm’s adaptive caps with the risk engine is small in this configuration ( $+4 [-0.1, +9.5]$ ).

### 7.3. RQ3: where reputation stops separating

A farmed-CAPTCHA attacker uses an address and fingerprint once and shares its country, prefix and residential ASN with every legitimate user, so

the penalty its failures earn lands on everyone: over an hour with caps lifted (five seeds), up to twice the legitimate rate 99 to 100% of it leaks, because a shared key stays above 0.3 conversion until failures outnumber legitimate sends about  $(0.8 - 0.3)/0.3 \approx 1.7$  to one, and the conversion penalty alone reaches only *delay*. At ten times, 70% leaks while 17% of legitimate users are challenged and 3.2% refused (claim C2, at least 90% leaked with caps lifted, held in 34 of 36 cells). Changing the risk score one setting at a time separates no better: a fresh-fingerprint weight of 40 stops the attacker only by challenging 47% of legitimate users and refusing 8.5%. In the settings run, reputation separated an attacker only on keys its sends dominated [13].

#### 7.4. RQ4: what rationing costs, and what it depends on

When separation fails, the cap is the lever: over ten seeds the default lets 47% of the farm’s requests through and refuses 41% of all legitimate users, the tightest setting 20% and 64%. Both assume an oracle job told the legitimate rate; run every five minutes or hourly instead of every minute, it lets 75 to 96% through. Learning the rate from the pipeline’s counters, the deployed job leaks 193 [161, 229] messages with a clean three-week profile, 340 [239, 441] more on a cold start, 106 [62, 151] more with a stale profile and 45 [10, 77] more with a poisoned one.

#### 7.5. Pumping against the sequential tests

Table 6: Concentrated pumpers on three blocks, caps lifted, ten seeds: messages leaked of about 574 requests in twenty minutes; seeds contained; mean minutes to containment (contained seeds) and to the first verdict (seeds with one); codes entered.

| Carrier, design                     | Leaked<br>(msgs) | Contained<br>(seeds) | Containment<br>(min) | First verdict<br>(min) | Codes<br>entered |
|-------------------------------------|------------------|----------------------|----------------------|------------------------|------------------|
| never verifies: v1                  | 570 [419, 736]   | 0/10                 | –                    | –                      | 0                |
| v2 default                          | 90 [71, 114]     | 10/10                | 3.5 [3.1, 3.9]       | 3.0                    | 0                |
| verifies within 1 s: v2 default     | 19 [17, 21]      | 10/10                | 1.4 [1.1, 1.7]       | 0.4                    | 19               |
| verifies 60% after 30 s: v2 default | 452 [365, 539]   | 0/10                 | –                    | 11.6                   | 272              |
| v2 default, 60 min                  | 1067 [839, 1310] | 2/10                 | –                    | –                      | –                |
| buys challenges: v2 default         | 105 [86, 127]    | 10/10                | 4.3 [3.8, 4.9]       | 3.0                    | 0                |

A pumper whose carrier never verifies is contained in all ten windows at minute 3.5 after 90 messages (Table 6); one verifying within a second is caught by the speed test at 19, and a challenge buyer leaks 105. The carrier verifying 60% with human-like delay draws its first verdict at minute 11.6 on

average; no window ends contained and 452 messages leak (1067 over sixty minutes, eight runs in ten never contained). Claim C3 (containment with at most 30% leaked) held in 36 of 36 cells.

Across 18 spread configurations leakage follows the blocks touched: with 30 blocks five to seven runs in ten are still contained. In the ten unsaturated configurations Equation (2) errs by 1.6 to 7.9 messages per run (1 to 13%); saturated, it overstates by up to 91.

## 7.6. Destination policies: where the counter wins, and where it fails

Table 7: Matched comparison, first protocol, 200 blocks (144 legitimate sends per block a day), tuning seeds 100–102, evaluation seeds 0–9: twenty-minute leakage (messages) by never-verifying (NV), instant (IV), human-like (HL) and challenge-buying (CB) carriers; from 24-hour attack-free runs at 65% conversion, completion, benign cost (points), users challenged and verdict events a day (none by construction for counters). Tuning target: 63.95%.

| Setting (selected as)                                           | NV  | IV  | HL  | CB  | Compl. | Benign | Chall. | Events |
|-----------------------------------------------------------------|-----|-----|-----|-----|--------|--------|--------|--------|
| none                                                            | 319 | 566 | 570 | 405 | 64.13% | –      | 1.53%  | 0      |
| sequential <i>T</i> 1000, <i>c</i> 1 (default)                  | 90  | 19  | 452 | 105 | 64.13% | 0.01   | 1.62%  | 5.8    |
| sequential <i>T</i> 300, <i>c</i> 0 (selected at tuning target) | 88  | 14  | 126 | 98  | 63.79% | 0.34   | 6.24%  | 314.3  |
| counter, graded, 4 per 10 min (selected)                        | 24  | 24  | 24  | 48  | 64.07% | 0.06   | 2.38%  | 0      |
| counter, graded, 160 per day (selected daily)                   | 319 | 409 | 408 | 405 | 64.11% | 0.02   | 1.87%  | 0      |

*At a benign service target.* Under the first protocol (Table 7, Figures 2 and 3), the sequential setting selected at the tuning target holds the human-like carrier to 126 messages only by raising 314 false verdict events a day, and falls 0.16 points below the target on seeds 0–9. The graded counter of four sends per block per ten minutes leaks 24 against each outcome-reading carrier (the default: 19 against the instant verifier) and 48 against the challenge buyer, at 0.06 points; counters selected at the sparser settings leak 6 to 18 at 0.15 to 0.35 points (up to 0.43 at 80%). With the full grid and density-specific attacks (E5) the rule selects the same counters, and so does every other objective we tried on the tuning runs over every setting meeting the target (worst-case leakage, a first-time-loss constraint, each attacker alone, 2000 random mixes), except that at 200 blocks the instant verifier alone, and 3 mixes, favour a sequential setting; on fresh seeds the counter still leaked least of the winners against every attacker (25, against 49 and 50, for the instant verifier). On seeds 300–309 the 1000-block counter falls 0.12 points below its target, yet no seed-and-conversion cell fails the paired check (the evaluation default minus 0.5 points).

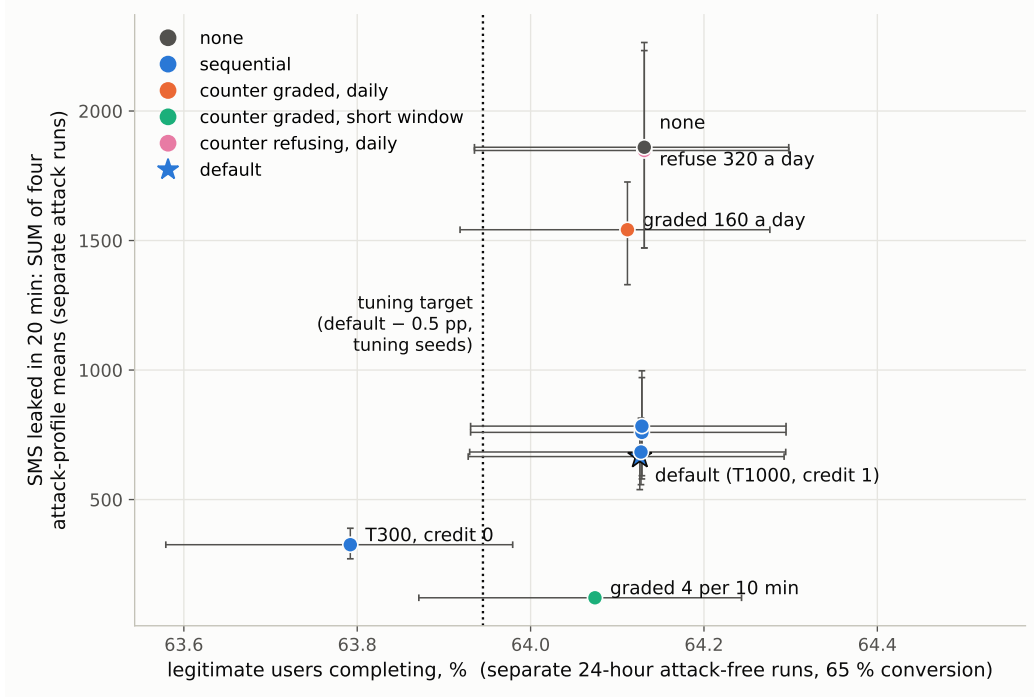


Figure 2: Matched comparison (first protocol, 200 blocks, seeds 0–9): each setting’s attack-free completion against leakage summed over the four pumpers’ means, with the tuning target.

Table 8: The second protocol’s claims for the principal counter (graded, 4 per 10 min) on the held-out family: 12 shifted points, held-out attack rates and pools, three seeds each, a 20-minute pumper on three blocks real users share. A claim holds if true in 90% of cells; cells share workloads and seeds and are not independent. IV: the instant verifier’s cells.

| Claim | Statement                                                             | True / cells   | IV    | Result                   |
|-------|-----------------------------------------------------------------------|----------------|-------|--------------------------|
| K1    | Leaks at most 10% of attack requests                                  | 144/144 (100%) | 36/36 | holds                    |
| K2    | Attack-free attributable loss at most 0.5 points                      | 36/36 (100%)   | –     | holds                    |
| K3    | Attributable loss under attack at most 0.5 points                     | 127/144 (88%)  | 32/36 | fails                    |
| K4    | Leaks no more than the default sequential tests                       | 131/144 (91%)  | 23/36 | holds pooled, not for IV |
| K5    | Attributable loss under attack at most the default’s plus 0.25 points | 93/144 (65%)   | 23/36 | fails                    |

315 *Under a second protocol.* Frozen and run on the held-out family (Table 8),  
316 the counter leaked 33, 34, 33 and 62 messages against the four carriers,

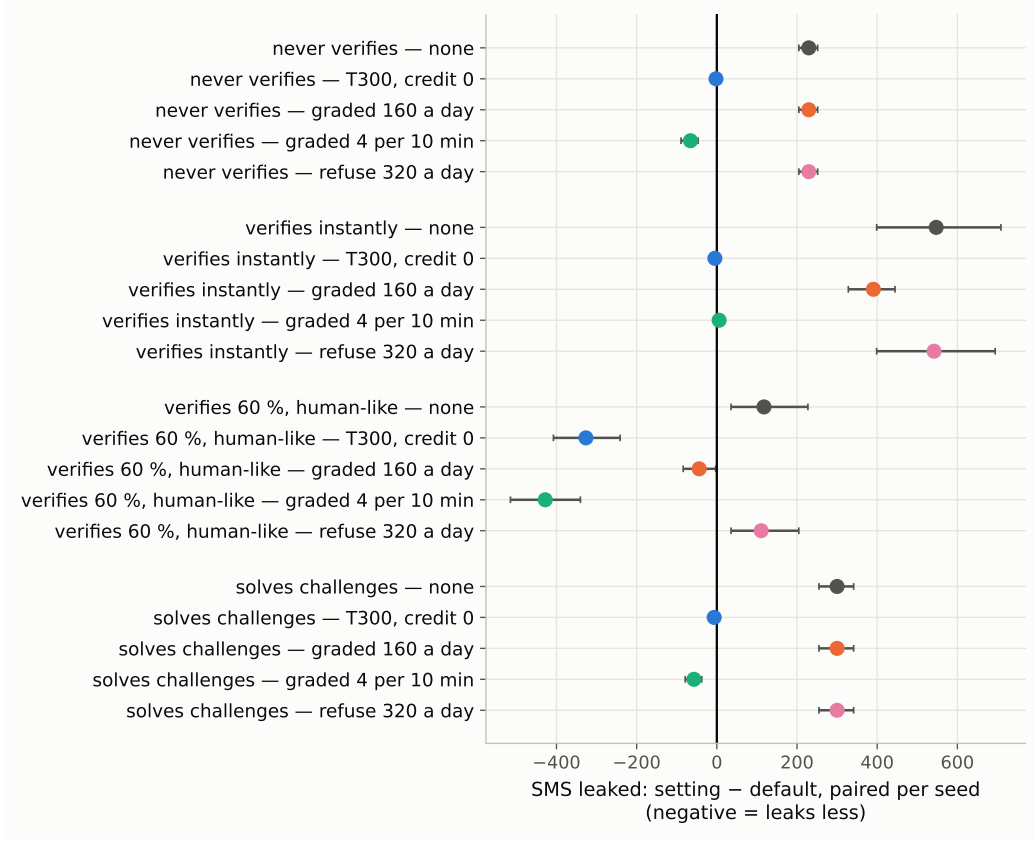


Figure 3: The same comparison: per-attacker paired differences from the default for the selected settings, with 95% intervals.

where the default leaked 287, 36, 344 and 287 and no policy 470 to 1857, at a benign cost of 0.13 [0.09, 0.17] points, and failed both service-under-attack claims: 13 of the 17 cells that broke K3 lie at the three points with the most legitimate traffic, where users and pumper compete for one quota; K5 failed in 51 cells over 11 of 12 points. K4 held in only 23 of the instant verifier's 36 cells. The counters selected for sparser traffic cost 4.7 and 5.8 points here even without an attack: the quota must match the traffic.

Table 9 separates three questions. Against the attacked no-policy arm the counter cost all users nothing on average and gained the attacked blocks' users 3 points on average, since without a destination policy the pumper exhausts their blocks and the hourly budget; with the human-like carrier alone it cost them 1.9. Against attack-free operation every evaluated policy



Table 9: Attacked service (E1): 200 blocks, 60 minutes, ten seeds. (a) A pumper on three blocks real users share: leakage summed and harm averaged over the four carriers, in points of all users and of the attacked blocks’ users (about 18 a run). (b) A poisoner on all 200 blocks, with 95% intervals. Per-carrier values and gross losses: artifact. Negative: a gain.

| (a) Four concentrated carriers      |        |                                |       |                               |       |        |
|-------------------------------------|--------|--------------------------------|-------|-------------------------------|-------|--------|
| Policy                              | Leaked | Extra harm<br>vs attacked none |       | Degradation<br>vs attack-free |       | Benign |
|                                     |        | all                            | block | all                           | block |        |
| none                                | 5 785  | –                              | –     | 0.19                          | 7.95  | –      |
| sequential, default                 | 1 164  | –0.09                          | –0.96 | 0.10                          | 6.99  | 0.01   |
| sequential <i>T</i> 300, <i>c</i> 0 | 397    | 0.12                           | 1.00  | 0.12                          | 8.95  | 0.19   |
| counter, 4 per 10 min               | 403    | –0.06                          | –3.16 | 0.07                          | 4.79  | 0.07   |
| counter + sequential                | 190    | –0.05                          | –3.05 | 0.07                          | 4.90  | 0.07   |
| token bucket, burst 4               | 424    | –0.13                          | –3.09 | 0.07                          | 4.86  | 0.00   |
| token bucket, burst 12              | 544    | –0.11                          | –2.19 | 0.08                          | 5.76  | 0.00   |

| (b) Block poisoner                  |        |                             |            |  |
|-------------------------------------|--------|-----------------------------|------------|--|
| Policy                              | Leaked | Extra harm vs attacked none |            |  |
|                                     |        | all                         | first-time |  |
| none                                | 1 976  | –                           | –          |  |
| sequential, default                 | 1 388  | 0.97 [0.52, 1.43]           | 1.22       |  |
| sequential <i>T</i> 300, <i>c</i> 0 | 871    | 2.45 [1.99, 2.90]           | 3.07       |  |
| counter, 4 per 10 min               | 1 662  | 0.72 [0.44, 1.02]           | 0.91       |  |
| counter + sequential                | 1 311  | 1.35 [0.85, 1.87]           | 1.70       |  |
| token bucket, burst 4               | 1 877  | 0.12 [0.04, 0.23]           | 0.16       |  |
| token bucket, burst 12              | 1 974  | 0.00 [0.00, 0.00]           | 0.00       |  |

329 still left those users 5 to 9 points worse off (8 with none): none shields  
330 a block an attacker shares. Counter and tests together, explored outside  
331 the frozen claims, leaked least against the four carriers (190 messages; the  
332 counter alone 403), but under the poisoner cost all users 1.35 points (the  
333 counter 0.72) and leaked 1 311, more than the tuned sequential setting (871).  
334 A token bucket with a burst of 4 cost the poisoner’s users 0.12 points only  
335 by letting 1 877 messages through; with two untuned settings, neither limiter  
336 dominates. Returning users, simulated as identities with verified history, lost  
337 nothing under these policies.

338 Attack-free (Figure 4a), the counter costs a block’s users nothing at one  
339 send per ten minutes, 2.3 points at four and 5.5 at eight, far below the ide-  
340 alized share beyond the quota (27% at four), since most challenged users  
341 complete or switch to WhatsApp; without WhatsApp eight sends cost 13.2,  
342 as users beyond twice the quota are refused. A 30-minute launch at three  
343 times the normal rate costs its blocks’ users 16 points. Over 60 minutes (Fig-

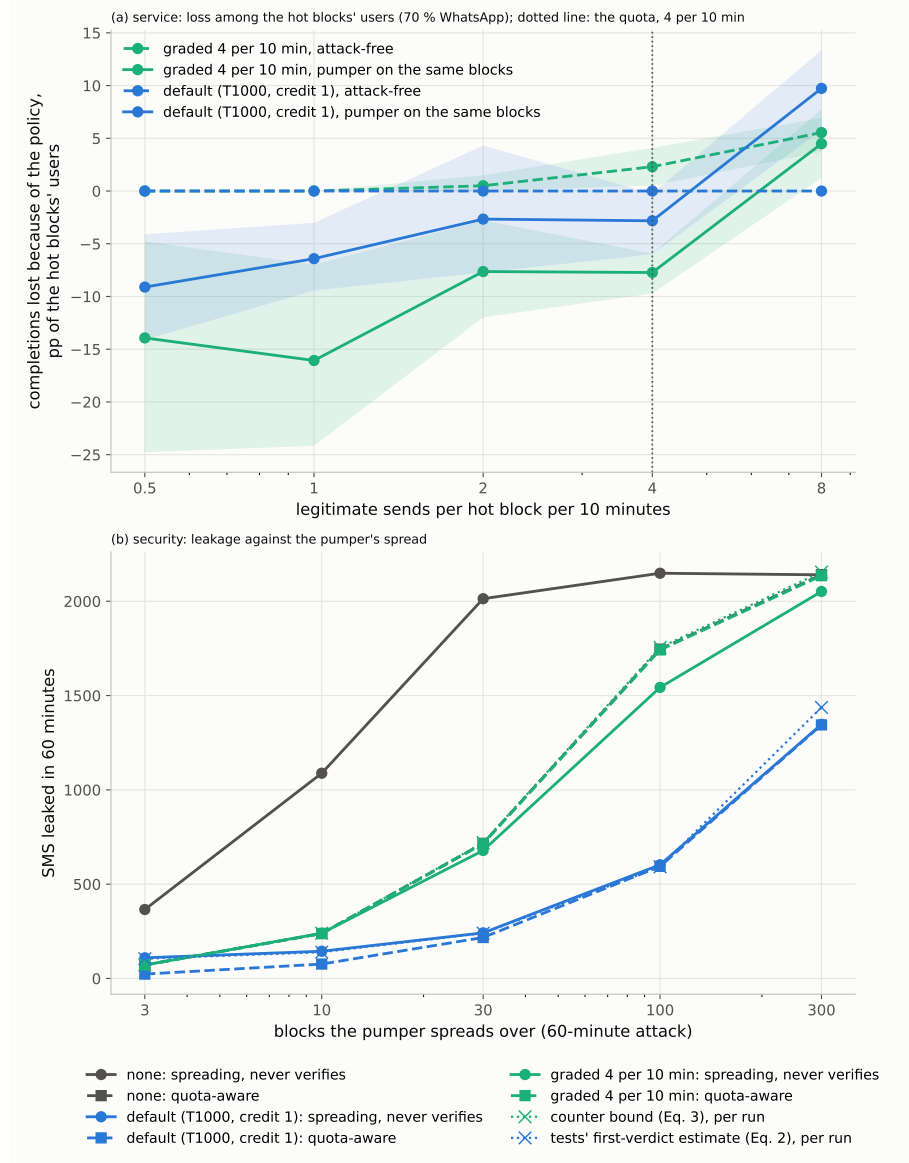


Figure 4: The counter's operating boundary (E4), five seeds; lines join sampled settings and do not locate a crossover. (a) Attributable loss in points of the three hot blocks' users (70% WhatsApp reachability) against their legitimate rate, without and with a never-verifying pumper on those blocks; bands: 95% bootstrap intervals; vertical line: the quota. (b) Mean leakage in 60 minutes against the blocks a pumper uses, at full rate (circles) or paced (squares); dotted: Equation (3)'s bound and Equation (2)'s first-verdict estimate for the full-rate pumper, from each run's offered requests and rate and the configured  $\tau$ , nothing fitted.

ure 4b) the default leaked 110, 144 and 242 messages on 3, 10 and 30 blocks, against 105, 139 and 240 from Equation (2) with  $\tau$  from the configuration and  $\lambda$  the offered rate ( $\lambda\tau \approx 90$ ), and the counter 71, 240 and 679 against its bound of 72, 240 and 720. With these unfitted inputs Equation (4) ordered the policies correctly in all 18 E4 cells where the estimates differ (3 to 300 blocks, 20 and 60 minutes, full-rate or paced; in two, both reach  $N$ ), and puts the crossover near five blocks for an hour and 30 for twenty minutes, between sampled spreads. A pumper paced to the quota leaks about as much under the counter as with no policy (71 against 72 on three blocks in an hour), and 23 under the tests. Against adaptive attackers (E3) the counter held the threshold-aware carrier and the receipt faker to 24 messages each (665 under the default) and the concentrated trust builder to 36 (502), but not trust builders on random numbers (577, as under the default); over 360 minutes the human-like carrier gained 81 messages an hour under the counter, 789 under the default.

#### 7.7. *RQ5: what destination policies cost legitimate users*

On one block, 20 000 Monte Carlo trials per cell give the probability of a verdict within 30 legitimate sends: 0.09% [0.06, 0.14] at 80% conversion with 20% autofill, 1.9% at 65% and 19.5% at 50%. Over 24 hours the default raises 0.5 verdict events a day on 200 busy blocks at 80% conversion and 5.8 at 65%, where they meet 27 of about 28 800 requests (claim C4 held in 18 of 18 cells).

A poisoner flooding for twenty minutes the twenty blocks legitimate users concentrate on (caps lifted, ten seeds) raises 16 verdict events that meet 98 legitimate requests, 33 never completing (descriptive); against the observe-only run on the same trace, enforcement costs 12.7 [4.6, 22.9] completions, 3.1% of users (6.3% without WhatsApp), and 23.9% with a 24-hour denylist. One that stops after ten minutes leaves verdicts that meet 406.5 requests afterwards, an attributable loss of 46.2 [16.9, 79.8] (48.2 over the whole run). Grading limits the harm; it caps it neither at a challenge nor at the attack's duration.

#### 7.8. *Adaptive attackers, economics and overhead*

These scripted adaptations' leakages are not bounds. Against v2, a datacenter attacker buying every challenge solution leaks 55; a threshold-aware carrier and a receipt faker leak 570 each, drawing no verdict.

379 If pumpers earn a share of the 0.1422 USD retail price and pay an event-  
 380 level bill, they break even at a share of 0.043 under v1 and 0.054 to 0.271  
 381 under v2 (the instant verifier 1.310); over 360 minutes under the counter the  
 382 human-like carrier needs 1.048. These are break-even shares under assumed  
 383 prices, not criminal profit, and a share above one under the counter says  
 384 nothing about spreading or exemption reuse.

385 On a real Redis (four vCPUs, four workers), a send costs 13.8 ms (me-  
 386 dian) and 21.1 ms (99th percentile). With the 400 ms floor and 50 ms vendor  
 387 calls, throughput reaches 133 requests per second at concurrency 128, with  
 388 30% of requests over the floor. With fast vendors no pair of five outcome  
 389 classes was distinguished (smallest Kolmogorov–Smirnov  $p = 0.051$ : no dif-  
 390 ference detected, not none; best single-threshold accuracy 53%). Under an  
 391 adversarial mixture with heavy-tailed vendors at concurrency 128, 81% of  
 392 sends overran the floor in server time, and a threshold on client response  
 393 times fitted to one run’s 2 497 sends and 22 refusals separated them with  
 394 88% balanced accuracy on that run, a resubstitution value; on three new  
 395 runs (two vCPUs, nearly every send over the floor), thresholds fitted on one  
 396 run scored 95 to 99.9% on the other two: a separation on that machine, not  
 397 a deployment accuracy.

## 398 8. Discussion

399 *Lessons.* Carriers that verify with human-like delay, fake receipts or know the  
 400 thresholds defeated our outcome tests but not a short-window counter, whose  
 401 allowance, unlike the tests’, grows with the windows an attack spans. Equa-  
 402 tion (4), a heuristic between the counter’s bound and the tests’ first-verdict  
 403 estimate, summarizes the trade-off for pumpers that solve no challenge, hold  
 404 no exemption and attack within a verdict’s hour. Its inputs, per-block load,  
 405 attack rate and spread, are in an operator’s logs, and  $\tau$  follows its own time-  
 406 outs; the cost of an excess send, competition under attack and exemption  
 407 abuse come only from the simulation. Counter and tests together, with a  
 408 quota set from each block’s traffic, is a hypothesis for real traffic, not a  
 409 recommendation.

410 *Limitations.* Everything is simulated and the calibration is assumed. Both  
 411 protocols were written by the author for the author’s generator, the second  
 412 knowing which counters had won. Some results follow from the generator (a  
 413 perfect premium list, a datacenter attacker that declines challenges, a farm

414 with human scores, returning users who always present verified history), and  
415 selection used an unweighted sum of four scripted pumpers’ leakage and a  
416 mean service target. Close rankings rest on ten or fewer seeds. Per-client  
417 quotas within a block, carrier spend ceilings, bounded exemptions and other  
418 sequential statistics lie outside the evaluated policies. Vendor adapters are  
419 untested live (App Attest enrolment only on synthetic certificate chains),  
420 and the timing result holds only for the observer model tested. Destination  
421 policies challenge people for what strangers sharing their block did, and de-  
422 ployment under the GDPR or Saudi Arabia’s PDPL needs a lawful basis and  
423 a route for refused users. Replaying real traffic is the experiment this paper  
424 most needs.

## 425 9. Conclusion

426 In this simulation the design contains attacks that carry a signature at lit-  
427 tle cost to simulated users, and rations, rather than separates, a residential at-  
428 tacker that looks like a person. Counting sends on the destination leaked less  
429 in aggregate than our outcome tests against concentrated pumpers, though  
430 not reliably against an instantly verifying carrier, and a heuristic from Sec-  
431 tion 5 ordered the two in all 18 boundary cells where its estimates differ; a  
432 frozen counter failed its service claims when the attacker shares blocks with  
433 real users, and lost its advantage against wide, paced and trust-building  
434 pumpers.

## 435 CRediT authorship contribution statement

436 **Naveed Ahmed:** Conceptualization, Methodology, Software, Valid-  
437 ation, Investigation, Writing – original draft, Writing – review & editing.

## 438 Declaration of competing interest

439 The author declares no known competing financial interests or personal  
440 relationships that could have appeared to influence this work.

## 441 Declaration of generative AI and AI-assisted technologies in the 442 writing process

443 The author used Claude (Anthropic) for code, evaluation, drafting and  
444 editing, checked numbers against the recorded results and claims against the  
445 code, and takes full responsibility for the content.

## 446 Data availability

447 No human or real traffic data was used. Code, simulation, protocols  
448 (`config/`) and every run's configuration and outcome (`results/`; per-seed  
449 counter outcomes in `counter_study_per_seed.csv`, sources of headline  
450 numbers in `headline_numbers.md`) are at  
451 <https://github.com/nahmedpsu/OTP-DDOS>. Release 2.8.0 (tag v2.8.0,  
452 commit 1c277a0) produced the results; releases 2.8.1 to 2.8.3 repair the  
453 defects of Section 6 and add the later analyses (`results/`  
454 `round5_analyses.md`, `condition_check.md`). These files are the  
455 supplementary material.

## 456 References

- 457 [1] MITRE Corporation, Resource hijacking: SMS pumping, technique  
458 T1496.003, MITRE ATT&CK, <https://attack.mitre.org/techniques/T1496/003/>, accessed 1 October 2026.
- 460 [2] Commsrisk, Elon Musk says Twitter lost \$60mn a year because 390  
461 telcos used bot accounts to pump A2P SMS, <https://commsrisk.com/elon-musk-says-twitter-lost-60mn-a-year-because-390-telcos-used-bot-accounts-to-pump-a2p-sms/>, trade press; the figure  
463 is the company's own claim (2023).
- 465 [3] J. H. Huh, H. Shin, S. Ahn, H. Yi, J. Cho, T. Kim, M. Lim, N. Choi,  
466 Preventing artificially inflated SMS attacks through large-scale traffic in-  
467 spection, in: 34th USENIX Security Symposium, USENIX Association,  
468 Seattle, WA, 2025.
- 469 [4] M. Sahin, A. Francillon, P. Gupta, M. Ahamad, SoK: Fraud in telephony  
470 networks, in: 2017 IEEE European Symposium on Security and Privacy  
471 (EuroS&P), IEEE, 2017. doi:10.1109/EuroSP.2017.40.
- 472 [5] M. Sahin, A. Francillon, Understanding and detecting international rev-  
473 enue share fraud, in: Network and Distributed System Security Symposi-  
474 um (NDSS), Internet Society, 2021. doi:10.14722/ndss.2021.24051.
- 475 [6] Twilio Inc., Verify fraud guard, [https://www.twilio.com/docs/veri-  
476 fy/preventing-toll-fraud/sms-fraud-guard](https://www.twilio.com/docs/verify/preventing-toll-fraud/sms-fraud-guard), product documenta-  
477 tion. Accessed 1 October 2026.

- 478 [7] Twilio Inc., SMS pumping risk score (Lookup v2), <https://www.twilio.com/docs/lookup/v2-api/sms-pumping-risk>, product documenta-  
479 tion. Accessed 1 October 2026.  
480
- 481 [8] M. Motoyama, K. Levchenko, C. Kanich, D. McCoy, G. M. Voelker,  
482 S. Savage, Re: CAPTCHAs – understanding CAPTCHA-solving ser-  
483 vices in an economic context, in: 19th USENIX Security Symposium,  
484 USENIX Association, 2010.
- 485 [9] A. Dionysiou, E. Athanasopoulos, SoK: Machine vs. machine – A sys-  
486 tematic classification of automated machine learning-based CAPTCHA  
487 solvers, *Computers & Security* 97 (2020) 101947. doi:10.1016/j.cose  
488 .2020.101947.
- 489 [10] D. Dean, A. Stubblefield, Using client puzzles to protect TLS, in: 10th  
490 USENIX Security Symposium, USENIX Association, 2001.
- 491 [11] X. Mi, X. Feng, X. Liao, B. Liu, X. Wang, F. Qian, Z. Li, S. Alrizah,  
492 L. Chen, Y. Qian, Resident evil: Understanding residential IP proxy  
493 as a dark service, in: 2019 IEEE Symposium on Security and Privacy  
494 (S&P), IEEE, 2019. doi:10.1109/SP.2019.00011.
- 495 [12] P. Richter, F. Wolsing, A. Chandrasekaran, R. Padmanabhan, M. All-  
496 man, R. Bush, A. Feldmann, A multi-perspective analysis of carrier-  
497 grade NAT deployment, in: *Proceedings of the 2016 ACM Internet  
498 Measurement Conference (IMC)*, ACM, 2016. doi:10.1145/298744  
499 3.2987474.
- 500 [13] J. R. Douceur, The Sybil attack, in: *Peer-to-Peer Systems (IPTPS  
501 2002)*, Vol. 2429 of LNCS, Springer, 2002, pp. 251–260. doi:10.100  
502 7/3-540-45748-8\_24.
- 503 [14] A. Wald, Sequential tests of statistical hypotheses, *The Annals of Math-  
504 ematical Statistics* 16 (2) (1945) 117–186. doi:10.1214/aoms/1177731  
505 118.
- 506 [15] E. S. Page, Continuous inspection schemes, *Biometrika* 41 (1/2) (1954)  
507 100–115. doi:10.2307/2333009.
- 508 [16] V. V. Veeravalli, T. Banerjee, Quickest change detection,  
509 arXiv:1210.5552 (book chapter) (2012).

